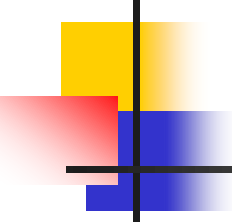


Duomenų bazių valdymo sistemos

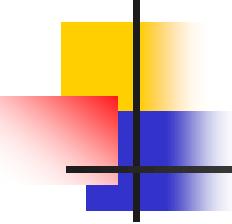
(įvadas, sąvokos)

prof. dr. Povilas Treigys

Povilas.Treigys@mif.vu.lt



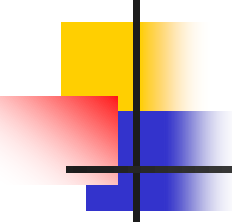
Duomenų bazės,
Duomenų bazių valdymo sistemos ir kt.



Duomenų bazės

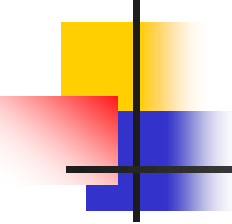
- Duomenų baze (**DB**) galima vadinti rinkinį tarpusavyje susijusių duomenų (dokumentų)

Vardas	Pavardė	Gimimo data	Studijų programa	Pakopa	Kursas
Jonas	Baltakis	1983.03.05	Informatika	Bakalaurantas	3
Ona	Gražulytė	1983.04.15	Informatika	Magistrantas	1



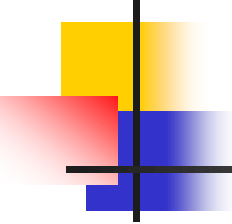
Duomenų bazės

- DB vartotojai gali atlikti tokius veiksmus:
 - Įtraukti naują įrašą
 - Modifikuoti esamą
 - Naikinti (trinti) įrašą
 - Vykdyti paiešką



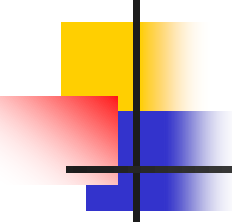
Duomenų bazės

- Programinė įranga manipuliuojanti duomenimis kartu su duomenų baze yra vadinama duomenų bazių sistema (**DBS**)
- DB vartotojai:
 - Asmenys tiesiogiai naudojantys DB
 - Taikomosios programos atliekančios operacijas su DB



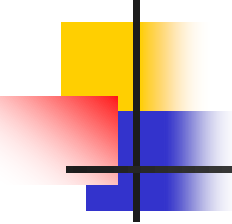
Duomenų bazės

- Duomenų bazių valdymo sistema (**DBVS**) – programinė įranga leidžianti vartotojui traktuoti DB kaip aukštesnio lygio objektą nei įrašų failas
- DBVS atlieka vartotojo sąsajos vaidmenį su DB
- Manipuliacija duomenimis atliekama naudojant formalią kalbą **SQL** (Structured Query Language)



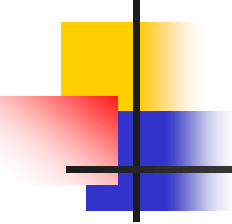
DBVS funkcijos ir keliami reikalavimai

- Minimizuoti duomenų perteklių (dubliavimą)
 - Popieriuje pateikti duomenys naudojami skirtingiems tikslams paprastai būna dubliuojami
 - Panaši situacija yra ir tada, kai duomenys yra saugomi ne tipizuotuose failuose
 - Dar blogiau, kiekviena taikomoji programa turi savo duomenų formatą



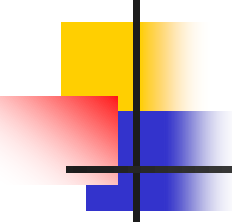
DBVS funkcijos ir keliami reikalavimai

- Be to, operacinė sistema atvėrus failą rašymui neleidžia kitiems vartotojams įrašinėti į jau atvertą failą, kol nebus baigtas rašymas ir failas bus uždarytas.
- Kadangi DB naudojasi daug vartotojų, tai DBVS turi užtikrinti veiksmingą DB naudojimą (visais atvejais)
 - Kai vienas vartotojas keičia įrašą, kiti tuo pat metu gali keisti kitus įrašus



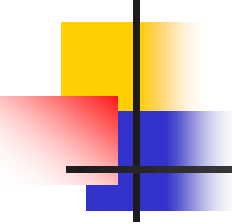
DBVS funkcijos ir keliami reikalavimai

- DB privalo išlaikyti tam tikras sąlygas keičiant, įtraukiant, ar šalinant duomenis
 - Studento bakalauro kursas negali būti didesnis nei 4, magistranto – 2.
- Taigi DBVS privalo užtikrinti DB duomenų vientisumą (integralumą)



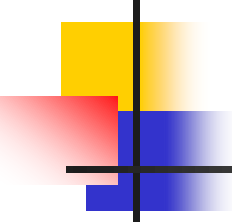
Duomenų integralumas

- Dvi grupės integralumų: fizinis ir loginis integralumai
 - Fizinis integralumas – užtikrinimas jog fiziškai duomenų bazė bus saugi, o duomenys joje korektiški (duomenų kopijos, atsitiktinės klaidos, ir kt.)
 - Loginis integralumas – užtikrinimas jog duomenys bus saugūs ir korektiški (šis užtikrinimas yra modelio lygyje)



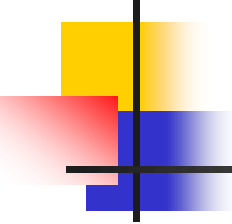
Loginis integralumas

- Esybių integralumas – apima objekto unikalius identifikatorius, esybių struktūrą, privalomus/neprivalomus laukus ar kt., pvz. neleidžia dubliuoti įrašų, ar pateikti įrašų kurių visi požymiai yra null.
- Ryšių integralumas garantuoja vienareikšmį dviejų objektų susiejimą skirtingose (o kartais ir toje pačioje) esybėje. Užtikrina galimybę manipuluoti susietais duomenimis, minimizuoja dubliavimą ir pan.
- Domeno integralumas - taisyklių rinkinys užtikrinantis galimybę įvesti tik tam tikrą galimų reikšmių poaibį.
- Naudotojo apibrėžtas integralumas – papildomas integralumas, kuriam pasiekti nepakanka E, R, D integralumų visumos.



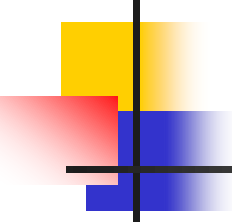
Saugumas kokybė ir integralumas

- Dažnai duomenų saugumas ir kokybė yra painiojami su integralumu:
 - Duomenų integralumas nėra duomenų saugumas.
 - Duomenų saugumas – pvz. taisyklių visuma apsauganti nuo neautorizuoto duomenų naudojimo.
 - Duomenų integralumas – duomenų korektiškumas tol kol duomenys yra saugomi. Taigi, duomenų saugumas yra vienas iš integralumo aspektų.
 - Duomenų saugumas nėra duomenų kokybė.
 - Duomenų kokybė yra duomenų saugomų DB atitiktis standartams ir procesams susijusiems su duomenų tikslumu, išsamumu, ir patikimumu.
 - Taigi, duomenų kokybė yra vienas iš duomenų integralumo aspektų.



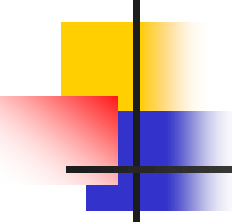
DBVS funkcijos ir keliami reikalavimai

- DBVS privalo užtikrinti duomenų neprieštarumą
 - Apie studentą yra du įrašai kur pirmame teigiama, kad jis studijuoja pirmame kurse, kitame – antrame.
 - Užklausos rezultatas priklausys nuo to, kuris įrašas ir iš kur bus paimtas.
- Tokia situacija nepasitaikytų jei studentui būtų skiriamas tik vienas įrašas (laukų grupių ribojimas)



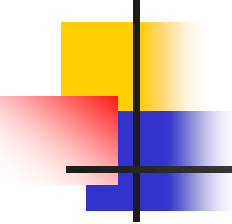
DBVS funkcijos ir keliami reikalavimai

- Saugumas (jau buvo užsiminta)
- Neplanuotų užklausų vykdymas labai didelėse DB



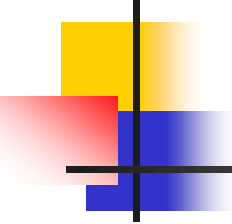
Reliacinės duomenų bazės

- DBVS suteikia vartotojui ne fizinę galimybę kreiptis į duomenų bazę, o leidžia operuoti tam tikromis sąvokomis
- Sąvokų rinkinys ir tai, kaip jos gali būti panaudojamos sudaro (nusako) **modelį**
- Iš čia – reliacinis (sąryšiais grįstas) duomenų modelis



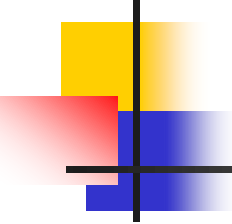
Reliacinės duomenų bazės

- DB kurios apibrėžiamos naudojant reliacinį modelį vadinamos reliacinėmis duomenų bazėmis (**RDB**)
- Atitinkamai valdymo sistemos – reliacinėmis duomenų bazių valdymo sistemomis (**RDBVS**)
- Modelis yra pasiūlytas Edgar Frank Codd 1970m.



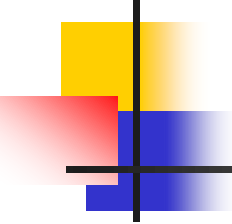
Reliacinės duomenų bazės

- Reliacinės DB sudaro eilę tarpusavyje susietų lentelių
- Reliacinio modelio pagrindas – lentelė, sudaryta iš:
 - Eilučių (kortežų, įrašų)
 - Stulpelių (atributas, laukas)
- Reliacinės sistemos principai:
 - Visi duomenys pateikiami lentelėmis
 - Pateikiami operatoriai generuojantys naujas lenteles iš senų (atskirų poaibių generavimas)



Reliacinės duomenų bazės

- Kiekviena lentelė turi unikalų pavadinimą
- Kiekviena jos eilutė **aprašo tam tikrą objektą, įvykį, situaciją ir kt.**
- Kiekvienas stulpelis – **vieną vienintelę objekto charakteristiką**
- Duomenų elementas arba reikšmė yra nusakoma stulpelio ir eilutės sankirta
- Stulpeliui priskiriamų reikšmių aibė yra vadinama **domenu**



Reliacinės duomenų bazės

1) Tiekėjų lentelė

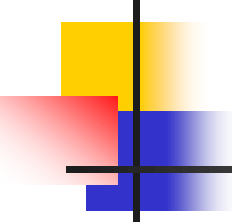
Tnum	Tpav	Statusas	Miestas
T1	Gabrys	20	Kaunas
T2	Jonaitis	15	Vilnius
T3	Antanėlis	10	Kėdainiai
T4	Burba	30	Utena
T5	Kedienė	20	Utena

2) Detalių lentelė

Dnum	Pavadinimas	Spalva	Svoris
D1	Varžtas	Juoda	3
D2	Sankaba	Mėlyna	9
D3	Medvaržtis	Raudona	11
D4	Veržlė	Žalia	4
D5	Sankaba	Raudona	13

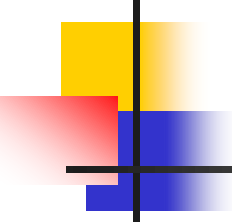
3) Tiekėjo patiektų detalių skaičiaus lentelė

Tnum	Dnum	Sk
T1	D1	9
T1	D2	4
T2	D1	3
T2	D2	8
T2	D6	9
T3	D2	13
T3	D3	6
T4	D3	7



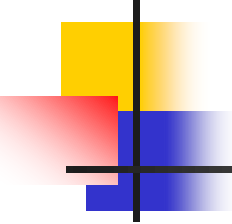
Reliacinės duomenų bazės

- Paprastai kiekviena lentelė turi savo pirminį raktą.
- Tai atributas arba grupė atributų, vienareikšmiškai nusakančių įrašą
 - Jis negali turėti nereikalingų atributų, t.y. išmetus iš pirminio rakto atributų nors vieną, likusių nepakaks vienareikšmiškam įrašo nusakymui
 - Pvz. mūsų lentelėse pirminiai raktai: *TNum*, *DNum*, ir pora atributų (*TNum* ir *DNum*), nes skaitoma, kad to paties tiekėjo tų pačių detalių įrašų nekartojame



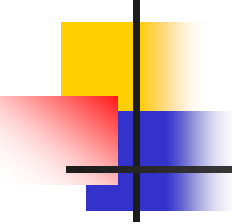
Kitos DB

- Hierarchinės – pateikiamos medžio struktūra
- Tinklinės – pateikiamos orientuotų grafų struktūra
- Deduktyviosios – hipotezių, faktų tikrinimui
- Ekspertinės – galinčios daryti išvadas atsižvelgiant į duomenis ir priimti sprendimus
- Objektinės – kiekvienas įrašas turi savo metodus ir atributus



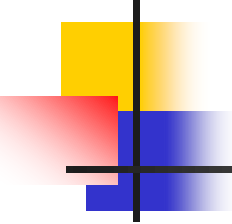
SQL

- SQL kalba paprastai yra naudojamas šiems uždaviniams spręsti:
 - DB kūrimui ir struktūros aprašymui
 - Informacijos išgavimui
 - DB saugumo kontrolės valdymui



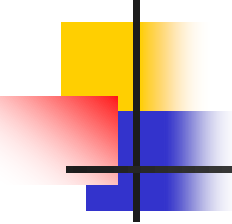
SQL

- SQL kalboje išskiriamos tokios pagrindinės sakinių grupės:
 - Duomenų apibrėžimo sakiniai (**DDL** – data definition language)
 - Duomenų apdorojimo sakiniai (**DML** – data manipulation language)
 - Duomenų valdymo sakiniai (**DCL** – data control language)



SQL

- DDL sakiniai naudojami DB bei lentelių kūrimui
 - Duomenų tipas yra domeno atitikmuo
- DML sakiniiais yra formuojamos užklausos (***query***) informacijai iš duomenų bazės išgauti
- DCL sakiniiais kontroliuojamas vartotojų darbas



SQL

- Formuojant SQL kalba užklausas reikia pateikti **KĄ** norime gauti, nekreipiant dėmesio į tai, kaip tai bus gauta.
- Už rezultato gavimą yra atsakinga DBVS
- SQL – deklaratyvi programavimo kalba (neprocedūrinė)
 - Deklaratyvus reiškia, kad teiginiais (deklaracijomis) aprašome, **KĄ** kažkuri programos dalis turi daryti, tuo tarpu imperatyviame programavime rašomos instrukcijos, kurios aprašo, **KAIP** programos dalys atlieka savo užduotis.

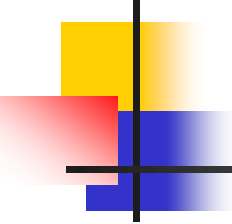


Duomenų bazių valdymo sistemos

Duomenų išvedimas

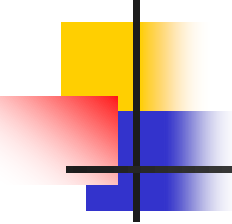
prof. dr. Povilas Treigys

Povilas.Treigys@mif.vu.lt



Paskaitos tema

- Duomenims iš DB išgauti yra naudojama SQL kalbos „SELECT“ sakiny
- Šio sakinio pagalba galima riboti išvedamos informacijos kiekį
- Gauti DB projekciją, atlikti išrinkimą ar lentelių sujungimą

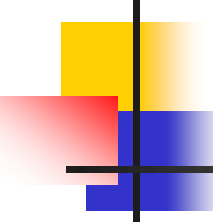


SELECT sakinio galimybės

- Projekcija – užklausos metu yra gražinamas tam tikras, norimas DB lentelės stulpelių poaibis
- Išrinkimas – tam tikro lentelės eilučių poaibio gražinimas po užklausos įvykdymo
- Sujungimas – duomenų iš kelių skirtingų lentelių gražinimas nustatant ryšį tarp jų

SELECT sakinio galimybės

Projekcija



Lentelė 1

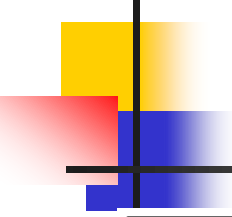
Išrinkimas

Lentelė 2

Sujungimas

Lentelė 1

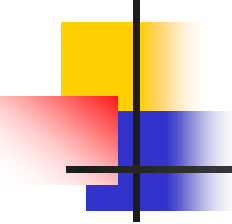
Lentelė 2



Bazinis SELECT sakiny

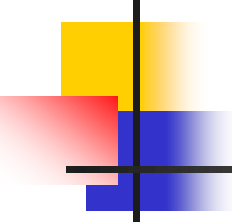
```
SELECT    * | { [DISTINCT] column | expression [alias], ... }  
FROM      table;
```

- SELECT – identifikuoja kokių stulpelių reikšmes norime išrinkti
- FROM – identifikuoja iš kokios lentelės norime išrinkti stulpelius, nurodytus po SELECT raktinio žodžio



Bazinis SELECT sakiny

- Privalomos sakinio dalys:
 - SELECT – nurodo išvedamus stulpelius
 - FROM – nurodo iš kurios lentelės
- Sintaksė:
 - SELECT – stulpeliai
 - * - išrenka visus stulpelius
 - DISTINCT – uždrausti pasikartojančius įrašus
 - *column/expression* – išrenka pateiktus stulpelius ar skaičiuoja tam tikrą išraišką
 - *alias* – suteikia išrinktiems stulpeliams kitus vardus
 - *FROM table* – iš kurios lentelės



Visų stulpelių išrinkimas

```
SELECT *  
FROM departments;
```

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting		1700

8 rows selected.

```
SELECT department_id, department_name, manager_id, location_id  
FROM departments;
```

Projekcijos poaibio formavimas

- SELECT galima naudoti norint išrinkti tik tam tikrus stulpelius iš lentelės
- Tam reikia po SELECT išvardinti stulpelių pavadinimus atskiriant juos kableliais

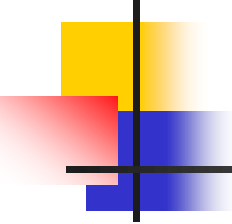
```
SELECT department_id, location_id  
FROM departments;
```

```
SELECT location_id, department_id  
FROM departments;
```

DEPARTMENT_ID	LOCATION_ID
10	1700
20	1800
50	1500
60	1400
80	2500
90	1700
110	1700
190	1700

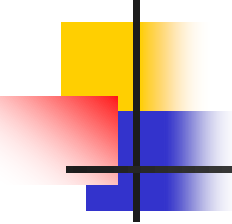
8 rows selected.





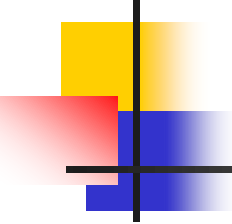
SQL sakinių rašymas

- SQL sakiniai gali būti rašomi tiek mažosiomis tiek didžiosiomis raidėmis
- SQL sakiniai gali būti rašomi keliose eilutėse
- SQL raktiniai žodžiai negali būti trumpinami ar jų dalys nukeliamos į kitas eilutes
- Sąlygos paprastai rašomos iš naujos eilutės



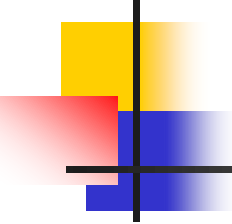
Aritmetinės operacijos

- Dažnai pasitaikanti situacija, kada saugomus duomenis reikia modifikuoti išvedimo metu, atlikti skaičiavimus ar tikrinti sąlygas “kas jeigu?”
- Tai įmanoma atlikti naudojant aritmetines išraiškas
- Aritmetinėse išraiškose gali būti naudojami stulpelių pavadinimai, konstantinės skaitinės reikšmės ir aritmetiniai operatoriai



Aritmetiniai operatoriai

- Su skaičiais ir datomis yra naudojami tokie operatoriai:
 - +
 - -
 - /
 - *
- Operatorių atlikimo eiliškumas:
 - * / + -
 - Daugyba ir dalyba atliekama pirmiausiai
 - To paties prioriteto operatoriai įvykdomi iš kairės į dešinę
 - Prioritetinę eilę galima keisti naudojant skliaustus



Aritmetinės operacijos (expression)

```
SELECT last_name, salary, salary + 300
FROM   employees;
```

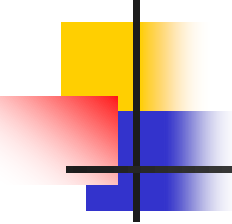
LAST_NAME	SALARY	SALARY+300
King	24000	24300
Kochhar	17000	17300
De Haan	17000	17300
Hunold	9000	9300
Ernst	6000	6300

...

Hartstein	13000	13300
Fay	6000	6300
Higgins	12000	12300
Gietz	8300	8600

20 rows selected.

- Pastaba – lauko „salary+300“ realiai DB nėra



Operatorių eiliškumas

```
SELECT last_name, salary, 12*salary+100
FROM employees;
```

LAST_NAME	SALARY	12*SALARY+100
King	24000	288100
Kochhar	17000	204100
De Haan	17000	204100
Hunold	9000	108100
Ernst	6000	72100

...

Hartstein	13000	156100
Fay	6000	72100
Higgins	12000	144100
Gietz	8300	99700

20 rows selected.

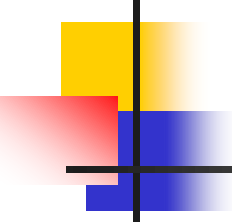
```
SELECT last_name, salary, 12*(salary+100)
FROM employees;
```

LAST_NAME	SALARY	12*(SALARY+100)
King	24000	289200
Kochhar	17000	205200
De Haan	17000	205200
Hunold	9000	109200
Ernst	6000	73200

...

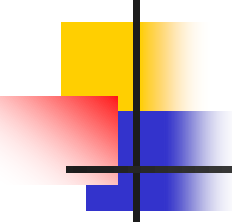
Hartstein	13000	157200
Fay	6000	73200
Higgins	12000	145200
Gietz	8300	100800

20 rows selected.



Null reikšmės

- Jei lentelės eilutės tam tikras stulpelio laukas neturi jokios reikšmės yra sakoma, kad tokio lauko reikšmė yra Null
- Null – nežinoma, nepriskirta, netaikoma ir t.t.
- Null nėra tas pat kas 0 ar ` `.
 - 0 – skaičius
 - ` ` – rašmuo, ženklas



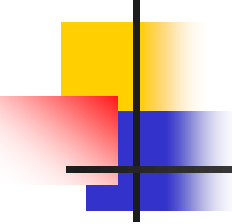
Null reikšmės

```
SELECT last_name, job_id, salary, commission_pct  
FROM employees;
```

LAST_NAME	JOB_ID	SALARY	COMMISSION_PCT
King	AD_PRES	24000	
Kochhar	AD_VP	17000	
...			
Zlotkey	SA_MAN	10500	.2
Abel	SA_REP	11000	.3
Taylor	SA_REP	8600	.2
...			
Gietz	AC_ACCOUNT	8300	

20 rows selected.

- Pastaba: Bet kokio domeno laukas gali įgyti Null reikšmės



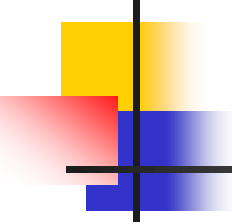
Null reikšmės

- Aritmetiniai reiškiniai į kuriuos įeina Null operandai taip pat įgyja reikšmę Null

```
SELECT last_name, 12*salary*commission_pct  
FROM employees;
```

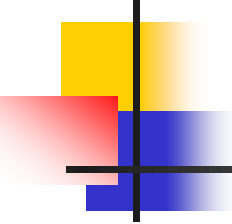
LAST_NAME	12*SALARY*COMMISSION_PCT
King	
Kochhar	
...	
Zlotkey	25200
Abel	39600
Taylor	20640
...	
Gietz	

20 rows selected.



Vardų suteikimas stulpeliams (*alias*)

- Stulpelio naujas vardas (*alias*):
 - Pakeičia saugomą stulpelio pavadinimą nauju
 - Naudojamas skaičiavimuose
 - *alias* vardas rašomas iš karto po stulpelio vardo palikus tarpą
 - Taip pat galima tarp stulpelio ir *alias* vardo rašyti raktinį žodį AS
 - *alias* vardą reikia rašyti tarp " " jei jame yra tarpai ar specialieji simboliai (#,\$)



Vardų suteikimas stulpeliams (alias)

```
SELECT last_name AS name, commission_pct comm
FROM employees;
```

NAME	COMM
King	
Kochhar	
De Haan	

...

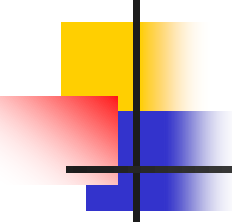
20 rows selected.

```
SELECT last_name "Name", salary*12 "Annual Salary"
FROM employees;
```

Name	Annual Salary
King	288000
Kochhar	204000
De Haan	204000

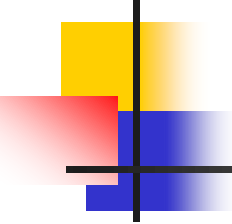
...

20 rows selected.



Konkatenacijos operatorius

- Apjungia stulpelius ar simbolių eilutes
- Yra žymimas `||`
- Sukuria stulpelį kurio elementai visada yra traktuojami kaip simbolių sąrašas



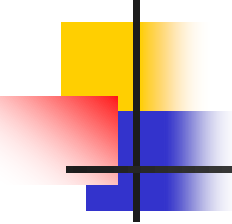
Konkatenacijos operatorius

```
SELECT  last_name||job_id AS "Employees"  
FROM    employees;
```

Employees
KingAD_PRES
KochharAD_VP
De HaanAD_VP
HunoldIT_PROG
ErnstIT_PROG
LorentzIT_PROG
MourgosST_MAN
RajsST_CLERK

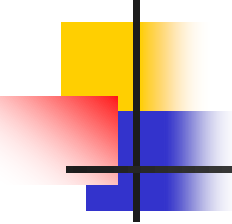
...

20 rows selected.



Pažodinės rašmenų (literals) eilutės

- Rašmuo yra simbolis, skaičius ar data įtraukti į SELECT'o atributų sąrašą
- Datos ir simbolių rašmenys turi būti rašomi tarp **apostrofų** ``
- Kiekviena rašmenų simbolių eilutė yra išspausdinama vieną kartą užklausos eilutės rezultate



Pažodinės rašmenų eilutės

```
SELECT last_name || ' is a ' || job_id  
       AS "Employee Details"  
FROM   employees;
```

Employee Details
King is a AD_PRES
Kochhar is a AD_VP
De Haan is a AD_VP
Hunold is a IT_PROG
Ernst is a IT_PROG
Lorentz is a IT_PROG
Mourgos is a ST_MAN
Rajs is a ST_CLERK

...

20 rows selected.

Pažodinės rašmenų eilutės

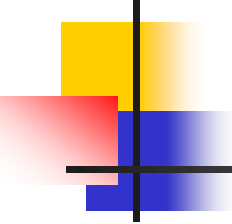
```
SELECT last_name || ': 1 Month salary = ' || salary Monthly  
FROM employees;
```

MONTHLY
King: 1 Month salary = 24000
Kochhar: 1 Month salary = 17000
De Haan: 1 Month salary = 17000
Hunold: 1 Month salary = 9000
Ernst: 1 Month salary = 6000
Lorentz: 1 Month salary = 4200
Mourgos: 1 Month salary = 5800
Rajs: 1 Month salary = 3500

...

20 rows selected.

- Kaip dar būtų galima perrašyti šią užklausą aiškiau, tačiau rezultatas liktų tas pats?



Pasikartojančios eilutės

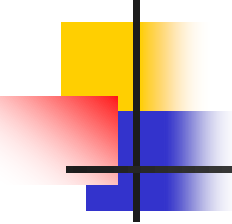
- Pagal nutylėjimą kiekviena užklausa pateikia rezultatus su visais pasikartojančiais įrašais esančiais DB

```
SELECT department_id  
FROM   employees;
```

DEPARTMENT_ID	
	90
	90
	90
	60
	60
	60
	50
	50
	50

...

20 rows selected.



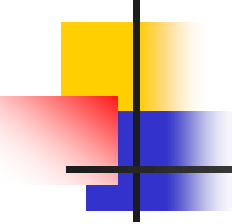
Pasikartojančios eilutės

- Norint išvengti pasikartojančių eilučių užklausos rezultate naudojamas raktinis žodis DISTINCT
- Šis žodis yra rašomas iškart po žodžio SELECT

```
SELECT DISTINCT department_id  
FROM employees;
```

DEPARTMENT_ID	
	10
	20
	50
	60
	80
	90
	110

8 rows selected.



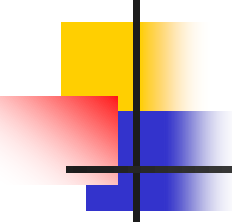
Pasikartojančios eilutės

- Po DISTINCT galima pateikti daugiau nei vieną stulpelio pavadinimą
- Šiuo atveju bus pateikta išvardytų stulpelių įgytų laukų reikšmių unikali kombinacija

```
SELECT  DISTINCT department_id, job_id  
FROM    employees;
```

DEPARTMENT_ID	JOB_ID
10	AD_ASST
20	MK_MAN
20	MK_REP
50	ST_CLERK
50	ST_MAN
60	IT_PROG
...	
	SA_REP

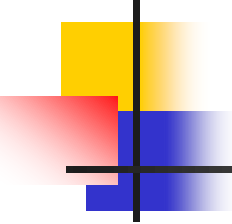
13 rows selected.



Lentelės struktūra

- Lentelės struktūrai pateikti yra naudojama:

```
DESC[RIBE] tablename
```

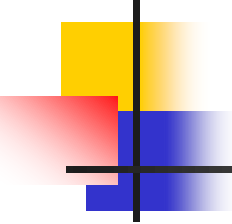


Lentelės struktūra

```
DESCRIBE employees
```

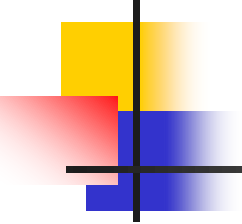
Name	Null?	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

- *Null?* – ar lauko reikšmė yra privaloma
- *Type* – domeno įgyjamų reikšmių aibė

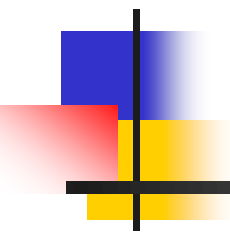


Duomenų tipai

- Ankstesnėje skaidrėje pateikto pavyzdžio domenai:
 - NUMBER (p,s) – skaičius, turintis p skaitmenų ir su s skaitmenų į dešinę nuo dešimtainio kablelio
 - VARCHAR2(s) – kintančio ilgio simbolinė reikšmė, kurios didžiausias ilgis gali būti s
 - DATE – data ir laikas intervale nuo sausio 1d. 1,4712m. pr. kr. g. iki gruodžio 31d. 9999m.
 - CHAR(s) – fiksuoto ilgio simbolių eilutė ne ilgesnė nei s



- Klausimai?



Duomenų bazių valdymo sistemos

Rūšiavimas, išrinkimo ribojimas

prof. dr. Povilas Treigys

Povilas.Treigys@mif.vu.lt

Eilučių ribojimas naudojant išrinkimą

EMPLOYEES

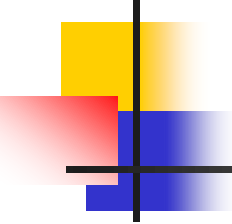
EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
100	King	AD_PRES	90
101	Kochhar	AD_VP	90
102	De Haan	AD_VP	90
103	Hunold	IT_PROG	60
104	Ernst	IT_PROG	60
107	Lorentz	IT_PROG	60
124	Mourgos	ST_MAN	50

...

20 rows selected.

**Gauti visus 90
departamento
darbuotojus**

EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
100	King	AD_PRES	90
101	Kochhar	AD_VP	90
102	De Haan	AD_VP	90

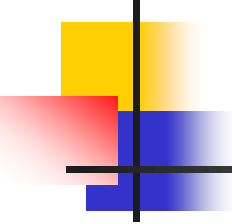


Išrinktų eilučių filtravimas

- Filtruoti vartotojui rodomas eilutes galima panaudojus raktinį žodį WHERE

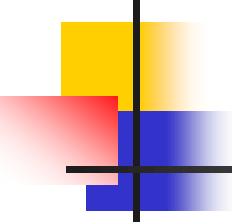
```
SELECT    * | { [DISTINCT] column | expression [alias], ... }  
FROM      table  
[WHERE    condition(s)];
```

- Raktinis žodis WHERE visada turi būti rašomas po žodžio FROM *tablename*



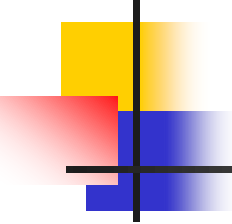
Išrinktų eilučių filtravimas

- Po WHERE raktinio žodžio visada seka sąlyga ar sąlygų rinkinys
- Jei sąlyga (os) įgyja reikšmę tiesa (true) tada tokia eilutė yra išspausdinama ekrane
- WHERE – apriboja užklausą tik tokioms eilutės kurios tenkina sąlygą
- *Condition(s)* – sąlyga ar jų rinkinys kurias gali sudaryti stulpelių pavadinimai, išraiškos, konstantos ar palyginimo operatoriai



Išrinktų eilučių filtravimas

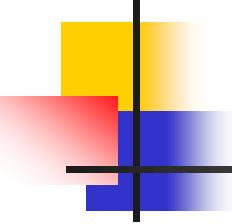
- WHERE skirsnis lygina reikšmes esančias stulpeliuose, aritmetines išraiškas ar funkcijas ir susideda iš:
 - Stulpelio pavadinimo
 - Lyginimo sąlygos
 - Stulpelio pavadinimo, konstantos ar reikšmių sąrašo



Išrinktų eilučių filtravimas

```
SELECT employee_id, last_name, job_id, department_id  
FROM   employees  
WHERE  department_id = 90 ;
```

EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
100	King	AD_PRES	90
101	Kochhar	AD_VP	90
102	De Haan	AD_VP	90



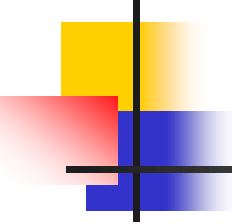
Simbolių eilutės ir datos

- Simbolių eilutės ir datos visada privalo būti rašomos tarp apostrofų (` `)
- Naudojant simbolines reikšmes didžiosios ir mažosios raidės yra skirtingai traktuojamos
- Naudojant datos žymenis svarbu formatas
 - Standartinis formatas yra DD-MEN-YY

```
SELECT last_name, job_id, department_id
FROM   employees
WHERE  last_name = 'Whalen';
```

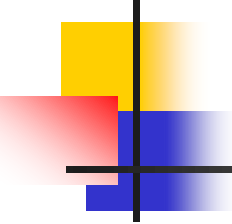
```
SELECT last_name, job_id, department_id
FROM   employees
WHERE  last_name = 'WHALEN';
```





Palyginimo operatoriai

- Palyginimo operatoriai SQL kalboje yra šie:
 - = - lygus
 - > - daugiau
 - >= - daugiau arba lygu
 - < - mažiau
 - <= - mažiau arba lygu
 - <> - nelygu



Palyginimo operatoriai

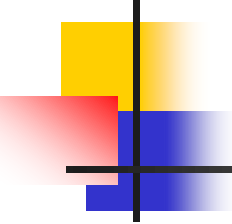
- Palyginimo operatorių sintaksė WHERE užklausoje yra:

-*WHERE išraiška operatorius reikšmė*

```
... WHERE hire_date='01-JAN-95'
```

```
... WHERE salary>=6000
```

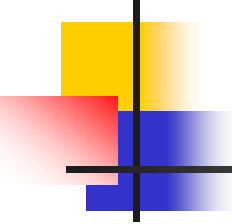
```
... WHERE last_name='Smith'
```



Palyginimo operatoriai

```
SELECT last_name, salary
FROM   employees
WHERE  salary <= 3000;
```

LAST_NAME	SALARY
Matos	2600
Vargas	2500



Kiti palyginimo operatoriai

- BETWEEN, ...AND... – tarp dviejų reikšmių jas įtraukiant
- IN(aibė) – bet kokio aibės elemento atitikimas
- LIKE – šablono atitikimas
- IS NULL – yra Null reikšmė

Sąlyga BETWEEN

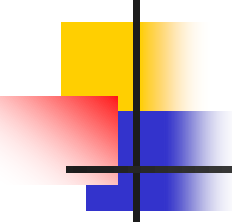
- BETWEEN naudojamas tada kada yra žinomi apatinis ir viršutinis intervalo rėžiai

```
SELECT last_name, salary
FROM   employees
WHERE  salary BETWEEN 2500 AND 3500;
```

↑
Apatinis rėžis

↑
Viršutinis rėžis

LAST_NAME	SALARY
Rajs	3500
Davies	3100
Matos	2600
Vargas	2500



Sąlyga IN

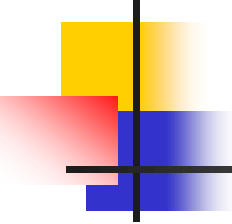
- Sąlyga IN naudojama tada kai reikia patikrinti ar DB lentelės eilutės reikšmė yra ieškomame sąraše

```
SELECT employee_id, last_name, salary, manager_id
FROM   employees
WHERE  manager_id IN (100, 101, 201);
```

EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
202	Fay	6000	201
200	Whalen	4400	101
205	Higgins	12000	101
101	Kochhar	17000	100
102	De Haan	17000	100
124	Mourgos	5800	100
149	Zlotkey	10500	100
201	Hartstein	13000	100

8 rows selected.

- *WHERE last_name IN ('Higgins','Hartstein');*



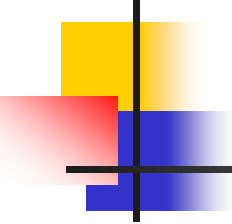
Sąlyga LIKE

- Ši sąlyga naudojama tada kai ieškoma pagal tam tikrą šabloną
- Paieškos sąlyga gali turėti tiek skaičius tiek simbolius
 - % - žymi nulį ar daugiau simbolių
 - _ - žymi vieną simbolį

```
SELECT    first_name
FROM      employees
WHERE     first_name LIKE 'S%';
```

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date LIKE '%95';
```

?

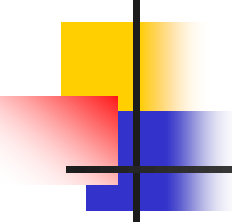


Sąlyga LIKE

- Galima šablonų kombinacija

```
SELECT last_name  
FROM   employees  
WHERE  last_name LIKE '_o%';
```

LAST_NAME
Kochhar
Lorentz
Mourgos



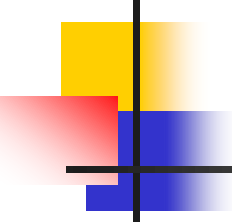
Sąlyga LIKE

- ESCAPE 'simbolis' - identifikatorius atitinka eilutės srauto išvedimo terminatorių

```
SELECT employee_id, last_name, job_id
FROM   employees
WHERE  job_id LIKE '%SA\_%' ESCAPE '\';
```

EMPLOYEE_ID	LAST_NAME	JOB_ID
149	Zlotkey	SA_MAN
174	Abel	SA_REP
176	Taylor	SA_REP
178	Grant	SA_REP

- Kokio šablono ieškoma?

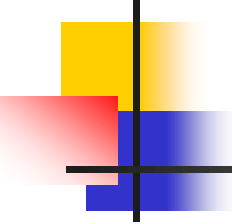


Sąlyga NULL

```
SELECT last_name, manager_id  
FROM   employees  
WHERE  manager_id IS NULL;
```

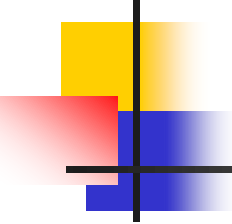
LAST_NAME	MANAGER_ID
King	

- IS NOT NULL – visos eilutės kurių stulpelių reikšmės yra žinomos



Kelių sąlygų sujungimas

- Labai dažnai pasitaiko kai vienos sąlygos norimai informacijai išgauti iš DB nepakanka, reikia sąlygas apjungti
 - AND
 - OR
 - NOT

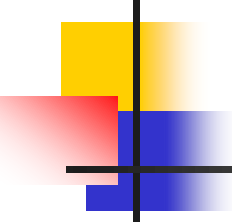


Operatorius AND

- Po WHERE žodžio visos užrašytos sąlygos privalo būti tenkinamos

```
SELECT employee_id, last_name, job_id, salary
FROM   employees
WHERE  salary >=10000
AND    job_id LIKE '%MAN%';
```

EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
149	Zlotkey	SA_MAN	10500
201	Hartstein	MK_MAN	13000



Operatorius OR

- Bent viena sąlyga turi būti tenkinama

```
SELECT employee_id, last_name, job_id, salary
FROM   employees
WHERE  salary >= 10000
OR     job_id LIKE '%MAN%';
```

EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
100	King	AD_PRES	24000
101	Kochhar	AD_VP	17000
102	De Haan	AD_VP	17000
124	Mourgos	ST_MAN	5800
149	Zlotkey	SA_MAN	10500
174	Abel	SA_REP	11000
201	Hartstein	MK_MAN	13000
205	Higgins	AC_MGR	12000

8 rows selected.

Operatorius NOT

- Neigimas

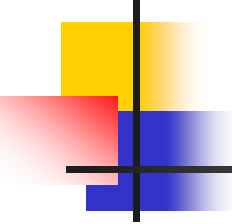
```
SELECT last_name, job_id
FROM employees
WHERE job_id
      NOT IN ('IT_PROG', 'ST_CLERK', 'SA_REP');
```

LAST_NAME	JOB_ID
King	AD_PRES
Kochhar	AD_VP
De Haan	AD_VP
Mourgos	ST_MAN
Zlotkey	SA_MAN
Whalen	AD_ASST
Hartstein	MK_MAN
Fay	MK_REP
Higgins	AC_MGR
Gietz	AC_ACCOUNT

10 rows selected.

- Teisinga:

```
... WHERE job_id NOT IN ('AC_ACCOUNT', 'AD_VP')
... WHERE salary NOT BETWEEN 10000 AND 15000
... WHERE last_name NOT LIKE '%A%'
... WHERE commission_pct IS NOT NULL
```



Operatorių eiliškumas

- Aritmetinės operacijos
- Konkatenacija
- Palyginimo operandai
- IS [NOT] NULL, LIKE, [NOT] IN
- [NOT] BETWEEN
- NOT
- AND
- OR

Eiliškumo pavyzdžiai

```
SELECT last_name, job_id, salary
FROM   employees
WHERE  job_id = 'SA_REP'
OR     job_id = 'AD_PRES'
AND    salary > 15000;
```

LAST_NAME	JOB_ID	SALARY
King	AD_PRES	24000
Abel	SA_REP	11000
Taylor	SA_REP	8600
Grant	SA_REP	7000

- Išrinkti eilutę, jei darbuotojas yra prezidentas ir uždirba daugiau nei 15000, arba jei darbuotojas yra pardavimų vadybininkas

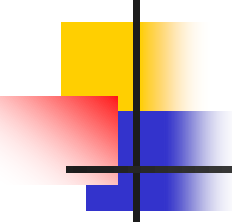
Eiliškumo pavyzdžiai

- Skliaustelių panaudojimas

```
SELECT last_name, job_id, salary
FROM   employees
WHERE  (job_id = 'SA_REP'
OR      job_id = 'AD_PRES')
AND    salary > 15000;
```

LAST_NAME	JOB_ID	SALARY
King	AD_PRES	24000

- Išrinkti eilutę, jeigu darbuotojas yra prezidentas arba pardavimų vadybininkas ir uždirba daugiau nei 15000



Išraiška ORDER BY

- Surūšiuoti užklausos eilutes galima naudojant ORDER BY
 - ASC – didėjimo tvarka
 - DESC – mažėjimo tvarka
- ORDER BY visada rašomas SELECT sakinio gale

Išraiška ORDER BY

```
SELECT last_name, job_id, department_id, hire_date
FROM employees
ORDER BY hire_date;
```

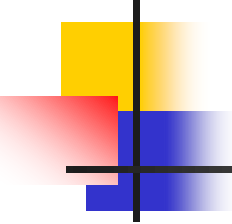
LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
King	AD_PRES	90	17-JUN-87
Whalen	AD_ASST	10	17-SEP-87
Kochhar	AD_VP	90	21-SEP-89
Hunold	IT_PROG	60	03-JAN-90
Ernst	IT_PROG	60	21-MAY-91

...

20 rows selected.

■ Sintaksė:

```
SELECT          expr
FROM            table
[WHERE          condition(s)]
[ORDER BY {column, expr} [ASC|DESC]];
```



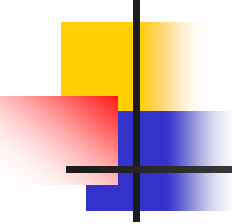
Rūšiavimas mažėjimo tvarka

```
SELECT last_name, job_id, department_id, hire_date
FROM employees
ORDER BY hire_date DESC ;
```

LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
Zlotkey	SA_MAN	80	29-JAN-00
Mourgos	ST_MAN	50	16-NOV-99
Grant	SA_REP		24-MAY-99
Lorentz	IT_PROG	60	07-FEB-99
Vargas	ST_CLERK	50	09-JUL-98
Taylor	SA_REP	80	24-MAR-98
Matos	ST_CLERK	50	15-MAR-98
Fay	MK_REP	20	17-AUG-97
Davies	ST_CLERK	50	29-JAN-97

...

20 rows selected.



Rūšiavimas pagal *alias*

```
SELECT employee_id, last_name, salary*12 annsal
FROM   employees
ORDER BY annsal;
```

EMPLOYEE_ID	LAST_NAME	ANNSAL
144	Vargas	30000
143	Matos	31200
142	Davies	37200
141	Rajs	42000
107	Lorentz	50400
200	Whalen	52800
124	Mourgos	69600
104	Ernst	72000
202	Fay	72000
178	Grant	84000

...

20 rows selected.

Rūšiavimas pagal keletą stulpelių

```
SELECT last_name, department_id, salary
FROM employees
ORDER BY department_id, salary DESC;
```

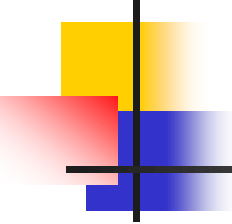
LAST_NAME	DEPARTMENT_ID	SALARY
Whalen	10	4400
Hartstein	20	13000
Fay	20	6000
Mourgos	50	5800
Rajs	50	3500
Davies	50	3100
Matos	50	2600
Vargas	50	2500

...

20 rows selected.

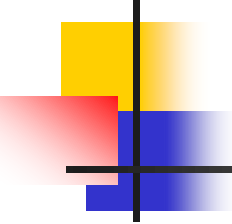
- Rūšiuoti galima ir pagal lauką kuris nėra SELECT išraiškoje:

```
SELECT last_name, salary
FROM employees
ORDER BY department_id, salary DESC;
```



Kintamųjų deklaravimas

- Oracle užklausų vykdymo metu galima pateikti parametrus
- parametrų nuskaitymui yra naudojami du operatoriai:
 - &
 - &&
- & operatorius reikšmę “pamiršta” iš kart po jos panaudojimo
- && operatorius atsimena (atsikratyri turime naudoti UNDEFINE komandą)



Kintamųjų deklaravimas

- `SELECT * FROM Employee Where job_id = '&darbo_kodas';`
- `SELECT * FROM Employee Where job_id = '&&darbo_kodas';`
- `UNDEFINE darbo_kodas;`